

# Evolutionary Computing 2024

Standard Assignment - Task 1: specialist agent - Group 12

Sarra Ben Ltaief  
Vrije Universiteit Amsterdam  
Student Number: 2830336  
Amsterdam, Netherlands  
s.benltaief@student.vu.nl

Nika Kovačić  
Vrije Universiteit Amsterdam  
Student Number: 2862120  
Amsterdam, Netherlands  
n.kovacic@student.vu.nl

Clàudia Domènech Farré  
Vrije Universiteit Amsterdam  
Student Number: 2742397  
Amsterdam, Netherlands  
c.domenech.farre@student.vu.nl

Emad Mshalwat  
Vrije Universiteit Amsterdam  
Student Number: 2658370  
Amsterdam, Netherlands  
e.mshalwat@student.vu.nl

## ACM Reference Format:

Sarra Ben Ltaief, Clàudia Domènech Farré, Nika Kovačić, and Emad Mshalwat. 2024. Evolutionary Computing 2024: Standard Assignment - Task 1: specialist agent - Group 12. In *Proceedings of September 12, 2024*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

---

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

*September 12, 2024, Amsterdam, Netherlands,*

© 2024 Copyright held by the owner/author(s).

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM.

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

## 1 INTRODUCTION

The EvoMan framework provides a robust environment for evaluating and comparing optimization algorithms. In the game simulation, the player competes against various enemies, and the goal is to defeat the enemy [1]. This project implements and tests two Genetic Algorithms (GAs) to optimize a player’s performance controlled by a neural network (NN). GAs are commonly used in Evolutionary Computing to optimize complex systems by simulating natural selection. The GA evolves a population of potential solutions by selecting the best performing individuals based on their fitness values, and then applying crossover and mutation to generate a new population. This process is repeated for a defined number of generations and enemies [2]. In this project, GAs are applied to evolve three specialist agents capable of defeating three enemies. The aim is to evaluate how different fitness functions influence the performance and adaptability of agents evolved through GAs. Two versions of the GA are compared, one that uses the default fitness function provided by the EvoMan framework, while the other introduces a custom fitness function that penalizes the agent for taking too long to eliminate the enemy. This is the basis of our research question:

**RQ:** How do the differences in fitness function design affect the evolution of agent behavior and performance in the EvoMan framework?

## 2 METHODS

### 2.1 Algorithm Description

We applied a GA to evolve a population of NN controllers for playing the EvoMan game. The algorithm begins by initializing a population of NNs, where each individual is represented by a weight vector. The algorithm iterates through a fixed number of generations. At each generation, individuals are evaluated using a fitness function that measures their performance against the enemy. This performance is tracked, and the best individual from each run is saved. After the fitness evaluation, individuals go throughout the reproduction process that includes three steps: selection, crossover and mutation. In addition, elitism was implemented to preserve the best solutions across generations. The results of the algorithm, including the best individual for each run and mean individual gains, are saved to files for further analysis. The fitness statistics across generations are also saved to provide insights into how the population evolves over time. In total, two versions of the GA were implemented, differing only in the design of the fitness function.

### 2.2 Experimental Setup

For both versions of the GA, experiments were run against three distinct enemies from the EvoMan environment: enemy 4, enemy 6, and enemy 7. Each of these enemies was selected for their distinct attack patterns. Enemy 4 has a possibility to enter a fireball state and rush towards the player. This requires the agent to learn to balance different attacks with defensive strategies. Next, enemy 6 attacks are based on distance. He jumps when the player gets too close or when the player shoots at him, forcing the agent to learn to control its distance and decide the next moves. Finally, Enemy 7 operates in a water environment, and uses two types of missiles,

one fired rapidly in horizontal trajectory, and the others moving in a sinusoidal way. The agent is required to adapt to dodging these different missiles (Miras K., 2019). The diversity in attack patterns of the three enemies allowed us to test the adaptability of our GAs.

Each experiment used a population size of 150 agents and was run for 20 generations. These values were chosen manually. In earlier stages of the development of our algorithm, we ran it with a population of 60 individuals and 10 generations. However, the results were significantly lower. From there, we systematically ran the algorithm in different values until we observed the best results. Later, the agents were evaluated using two different fitness functions, which are explained in the next section. Each experiment was repeated for 10 independent runs per enemy to ensure statistical significance and account for variability in performance. The difficulty level of the enemies was set to level 2, as specified in the guidelines, and the visuals were disabled to optimize computational efficiency. During hyperparameter tuning, optimal mutation rate value was explored. According to [3], optimal mutation rate was identified between 0.005 and 0.01. However, during the trials, the results of the fitness function were pretty low, meaning that agents did not perform well. The mutation rate of 0.2 was chosen as it resulted in best fitness values, alongside other parameters.

The same Genetic Algorithm parameters were used across all experiments. We started with a mutation rate of 0.2, and tournament selection, which ensures a balance between selecting strong individuals while maintaining population diversity. Single-point crossover was used for reproduction, and mutation was applied randomly to individual weights. The performance of each agent was evaluated based on its fitness score, which was tracked throughout the experiment. In addition, we also calculated the individual gain, defined as the difference between the player’s remaining energy and the enemy’s remaining energy at the end of the battle. This measure provides insight into the agent’s relative success in combat and will be explored in the boxplot analysis. At the end of each run, the best-performing agent from the final generation was saved.

### 2.3 Fitness functions

Two different fitness functions were used to evaluate the performance of agents during the evolution process, one for each version of the GA. The first version, GA1, used the default fitness function provided by the EvoMan framework, evaluating the agent’s performance as follows:

$$\text{fitness} = 0.9 \cdot (100 - e) + 0.1 \cdot p - \log(t)$$

In this formula,  $e$  represents the remaining energy of the enemy,  $p$  is the player’s remaining energy (both in a range of 0 to 100), and  $t$  is the time taken to defeat the enemy. The fitness function maximizes player energy while minimizing enemy energy and the time taken to complete the battle. The logarithmic time penalty ensures that faster eliminations are favored, but without excessively penalizing longer battles. For the second version, GA2, we introduced a custom fitness function that imposes a heavier penalty on time, prioritizing faster eliminations. The custom fitness function is defined as:

$$\text{fitness} = 100 - e - kxt$$

where  $e$  is the enemy's remaining energy,  $t$  is the time taken to defeat the enemy, and  $k$  is a constant that controls the time penalty. It penalizes agents that take too long to defeat the enemy. The penalty increases proportionally with time, forcing the agents to focus on both reducing the enemy's energy and optimizing the speed of elimination. Initially, we used a starting value of  $k=0.1$  but adjusted it to  $k=0.05$  after initial tests showed that the higher penalty disproportionately penalized agents for longer battles, even when they performed well. By lowering  $k$ , the fitness function still favors faster eliminations but allows for a more balanced evaluation of agent performance in longer fights.

## 2.4 Selection, Crossover and Mutation

Both versions of the GA use the same evolutionary operators to improve the agent's performance over generations. Tournament selection is applied with a tournament size of 5, which ensures that fitter individuals are more likely to reproduce while still maintaining diversity within the population. A single-point crossover is then used for reproduction where a random point in the genome is selected, and the genetic material is split at that point [2].

A mutation rate of 0.2 is applied in order to introduce variability. We selected this value as it maintains a balance between keeping diversity in the population as suggested in [3]. Mutation occurs when a random value drawn for each weight is smaller than the mutation rate. When this happens, a small random value between -0.1 and 0.1 is added to the individual weights in order to add exploration of new regions in the solution space, helping to avoid the algorithm becoming stuck in local optima.

Additionally, elitism is used to carry over the top two individuals from each generation into the next without modification. With this method, we are keeping the best solutions and preventing their loss due to the randomness introduced by crossover and mutation.

## 3 RESULTS

The progression of both average fitness and maximum fitness was tracked across 20 generations for each of the three enemies: Enemy 4, Enemy 6, and Enemy 7. Figure 1 illustrates the overview of the results.

*Enemy 4:* In GA1, the mean maximum fitness shows a rapid increase during the first 5 generations, reaching a plateau at 85, while the mean fitness rises more slowly, stabilizing at 60 by generation 20. In contrast, in GA2, both curves progress more slowly, with the maximum fitness stabilizing at 60 by generation 20, suggesting that the time penalty in GA2 may have slowed the agents' convergence toward optimal solutions, as they focused more on faster eliminations, potentially limiting more balanced strategies. The variance across runs is indicated by the shaded regions, showing a considerably smaller variance in the mean max fitness for GA1, suggesting more consistent solutions with that algorithm.

*Enemy 6:* In GA1, the fitness progression shows a rapid improvement during the initial generations, for both curves. By generation 8, the curves stabilize, showing that the algorithm has converged to an optimal solution early in the process, suggesting that it benefits from the fitness function's balanced focus on survival and enemy energy reduction. However, for GA2, the fitness progression shows

a slower increase, suggesting that the time penalty in GA2 may have made it more challenging for agents to develop optimal strategies quickly, as they need to balance the speed of eliminations with survival. However, the final values are similar in both GA1 and GA2 for Enemy2, showing that both algorithms can reach similar final performance.

*Enemy 7:* In both GA1 and GA2 a rapid increase in both curves can be observed during the first 6 generations, stabilizing in generation 10. However, we see a more consistent improvement in GA2 in mean fitness across the generations ending slightly higher by generation 20. It is not a significant difference but could suggest that the time penalty in GA2 promotes more efficient strategies based on faster eliminations. Overall, both algorithms stabilize with a mean curve at the same value (fitness = 80), showing similar final performance.

The boxplot for enemy 4 shows that GA1 produces higher individual gains with a median around 70 and a significantly smaller spread, indicating more consistent performance across runs. In contrast, GA2 shows lower individual gains, with a median close to zero and a much wider range, suggesting greater variability in the agents' performance. The time penalty in GA2 likely discouraged defensive strategies, resulting in less effective overall performance compared to GA1.

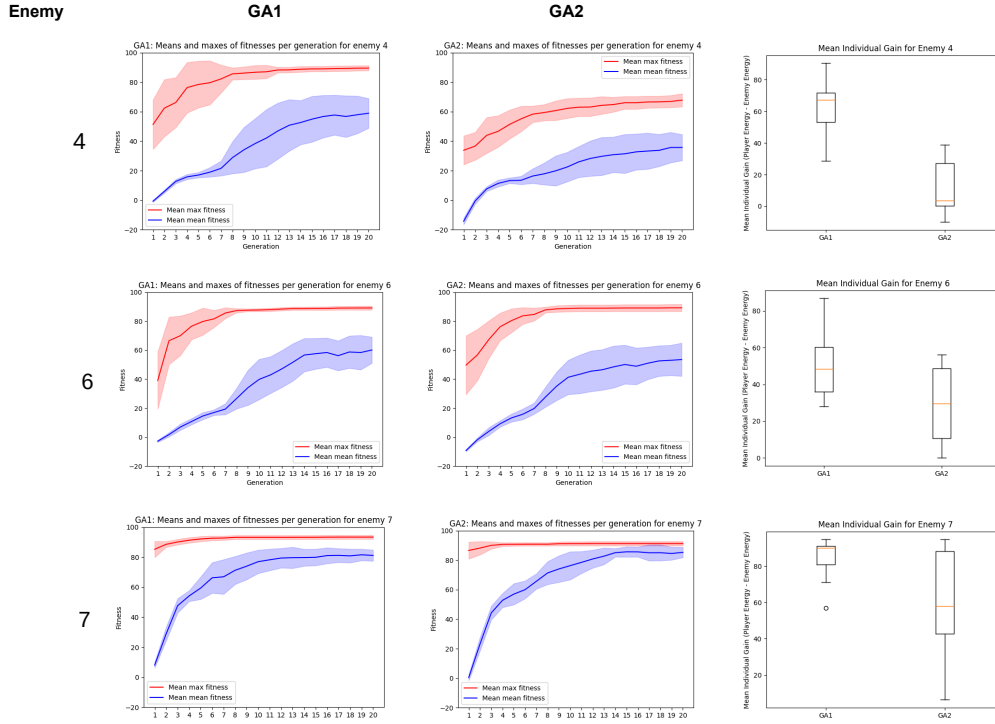
For Enemy 6, GA1 again shows higher individual gains, with a median around 50 and a tighter interquartile range, demonstrating more consistent performance. GA2 presents a lower median gain around 25, with greater variability. The results suggest that GA1's balanced fitness function allowed agents to evolve more stable strategies, while GA2's emphasis on speed led to more erratic results and less overall gain.

The results for Enemy 7 are notably different. Here, GA2 demonstrates a wider range of individual gains, with some agents performing significantly well, as indicated by the higher spread and median compared to GA1. GA1, though more consistent, shows lower gains overall. This suggests that GA2's fitness function, which prioritizes faster eliminations, may be more effective against Enemy 7, where speed has a bigger influence in success.

## 4 DISCUSSION

In this project, the main aim was to explore how the design of fitness functions influences the evolution of agent behavior and performance within the EvoMan framework. The results show important insights between strategy adaptability and performance consistency.

The choice of fitness function directly influenced the behavior of agents within the two different algorithms. On the one hand, GA1 demonstrated more consistent and higher performance across all enemies. This was illustrated in the fitness progression plots where GA1 agents consistently reached higher mean and maximum fitness levels, particularly for Enemy 4 and Enemy 6. The faster convergence in GA1 shows that its fitness criteria caused agents to try different strategies, helping them perform well early in the evolution process. On the other hand, GA2 imposed a heavy penalty for time, which led to slower convergence and lower overall fitness. This suggests that the time penalty forced agents to prioritize



**Figure 1: An overview of results.** The results for each enemy are displayed by the row. The first and second column contain line plots of the mean and max fitness values at each generation, averaged for the 10 runs for GA1 and GA2 respectively. The third column contains boxplots, showing the results of the mean over 5 trials of the best agent in each run.

rapid eliminations over long-term survival and energy preservation, limiting their ability to explore different strategies.

The individual gain results further illustrate the difference between the two algorithms. GA1 achieved higher performance across all enemies, indicating that its agents were better at balancing offense and defense. The smaller variance in GA1’s results suggests more stable and adaptable strategies, with consistent performance across different runs. However, GA2’s focus on time led to more unpredictable outcomes. While GA2 agents showed higher variability, especially against Enemy 7, their overall performance was less consistent. This could be due to the aggressive strategies encouraged by the time penalty.

We want to emphasize that the greater variability in G2 caused a peak performance in certain moments for G2, particularly against enemy 7. This suggests that even though GA2’s time penalty generally encouraged aggressive, inconsistent strategies, it occasionally produced highly effective solutions in scenarios where rapid limitations were required.

Our results demonstrated how fitness function design significantly affects the performance of agents evolved through GAs. GA1, with its balanced fitness function, led to faster convergence and more consistent performance. In contrast, GA2’s time penalty pushed agents toward aggressive, speed-focused strategies, resulting in greater variability and less stability. While GA2 occasionally excelled in situations requiring quick eliminations, GA1 overall provided more reliable and adaptable solutions, highlighting the

importance of aligning fitness functions with the specific goals of the task.

## 5 CONCLUSIONS

This project highlighted the impact of fitness function design on the evolution of agents in the EvoMan framework. While GA1 provided more reliable and adaptable strategies, GA2 showed potential in speed-focused scenarios. These insights reinforce the need for careful consideration when designing fitness functions for evolutionary algorithms. Future research could explore adaptive fitness functions that adjust dynamically during different stages of evolution—for example, focusing on exploration in early generations and shifting to exploitation as agents near optimal solutions. Hybrid approaches could also be developed to balance speed and stability, allowing agents to perform well under different conditions by combining penalties for time and rewards for strategic depth, resulting in more versatile and adaptable agents.

## REFERENCES

- [1] Karine da Silva Miras de Araújo and Fabrício Olivetti de França. 2016. An electronic-game framework for evaluating coevolutionary algorithms. *arXiv preprint arXiv:1604.00644* (2016).
- [2] Agoston E Eiben and James E Smith. 2015. *Introduction to evolutionary computing*. Springer.
- [3] VP Patil and DD Pawar. 2015. The optimal crossover or mutation rates in genetic algorithm: A review. *International Journal of Applied Engineering and Technology* 5, 3 (2015), 38–41.